PropOS 多设备登录认证机制设计报告

文档版本：v1.0

日期：2026-04-21

模块：后端认证（M0 / 基础设施层）

状态：现行设计

---

1. 概述

PropOS 认证系统支持同一账号在 Flutter App（iOS/Android/HarmonyOS Next）、uni-app（小程序/H5/HarmonyOS）、Admin Web 三端**同时登录、独立运行**，互不干扰。

核心设计思路：无状态 JWT（Access Token）+ 有状态旋转刷新（Refresh Token），通过数据库 `refresh_tokens` 表按行粒度管理每台设备的会话状态。

---

2. 技术架构

2.1 Token 体系

Token 类型	格式	有效期	存储位置
Access Token	HS256 JWT	由 `JWT_EXPIRES_IN_HOURS` 配置（典型值 1~24h）	客户端内存 / 安全存储
Refresh Token	随机不透明字符串（SHA-256 哈希后落库）	30 天	客户端持久存储

2.2 各端 Token 存储

客户端	Access Token	Refresh Token
Flutter App	`flutter_secure_storage`（Keychain/Keystore）	`flutter_secure_storage`
uni-app	`uni.getStorageSync`（小程序/App 本地沙箱）	`uni.getStorageSync`
Admin Web	内存（`axios` 拦截器）	`localStorage`

三端存储完全物理隔离，互不读写。

2.3 服务端数据模型

`refresh_tokens` 表核心字段：

```
CREATE TABLE refresh_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  token_hash  TEXT NOT NULL UNIQUE,   -- SHA-256(原始 token)，明文不落库
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked     BOOLEAN NOT NULL DEFAULT FALSE,
  device_info TEXT,                   -- 设备标识（可选）
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

同一 `user_id` 可存在多行（对应多个设备），每行独立管理。

---

3. 关键流程

3.1 多设备并发登录

每次 `POST /api/auth/login` 调用，后端向 `refresh_tokens` 插入一条新记录，不撤销已有记录：

```
Flutter App 登录 → 插入行 A（user_id=U, token_hash=H_app）
uni-app 登录   → 插入行 B（user_id=U, token_hash=H_uni）
Admin 登录     → 插入行 C（user_id=U, token_hash=H_web）
```

三行并存，互不影响。

3.2 单设备退出（Token 粒度撤销）

`POST /api/auth/logout` 处理流程：

```
客户端传 raw_refresh_token
  → SHA-256(raw_refresh_token) 得 hash
  → 查 refresh_tokens WHERE token_hash = hash
  → revoke(stored.id)  ← 仅撤销该行，其余行不动
  → 写 audit_logs（USER_LOGOUT）
```

uni-app 退出仅撤销行 B，行 A（Flutter）和行 C（Admin）保持 `revoked=false`，继续有效。

3.3 Refresh Token 旋转刷新

`POST /api/auth/refresh` 处理流程：

```
客户端传旧 refresh_token
  → 查库验证（未撤销、未过期）
  → revoke(旧 token 行)      ← 撤销旧行
  → 签发新 access_token + 新 refresh_token  ← 插入新行
  → 返回新令牌对
```

旋转机制防止 Refresh Token 重放攻击，每次刷新后旧 token 立即失效。

3.4 全端强制下线

以下操作触发 `revokeAllForUser(userId)` ，撤销该用户所有设备的 Refresh Token：

触发场景	触发方	效果
用户修改密码（ `POST /api/auth/change-password` ）	用户主动	所有端 Refresh Token 失效
管理员冻结账号	后台操作	`session_version` 自增，Access Token 也同步失效
管理员停用账号	后台操作	同上

3.5 session_version 双重验证

除签名验证外，每次请求都会与数据库比对 `session_version` ：

```
JWT payload.session_version ≠ users.session_version
  → 返回 TOKEN_REVOKED(401)
  → 强制客户端重新登录
```

此机制确保改密/冻结后，即使 Access Token 未过期，也会在下一次请求时立即被拒绝。

---

4. 安全机制清单

机制	实现细节	防御场景
HS256 强制签名	`_enforceHS256Algorithm(token)` 在验签前检查 header	算法混淆攻击（alg:none / RS256 替换）
session_version 校验	每次请求查库比对	改密/冻结后旧 JWT 继续通行
bcrypt cost=12	`BCrypt.hashpw(..., logRounds: 12)`	暴力破解密码哈希
时序攻击防护	用户不存在时执行假 bcrypt 比较	通过响应时间差枚举账号存在性
Refresh Token 哈希存储	SHA-256(raw token)，明文不落库	数据库泄露后 token 无法直接使用
Refresh Token 旋转	旧 token 刷新后立即撤销	Refresh Token 重放攻击
登录失败锁定	连续失败 ≥5 次锁定 30 分钟	暴力破解登录
登出审计日志	`audit_logs` 记录每次 USER_LOGOUT	操作审计 / 异常溯源
改密审计日志	`audit_logs` 记录每次 change-password	操作审计 / 合规要求

---

5. 设计优缺点评估

5.1 优点

多端体验：手机退出不踢掉 PC 后台，符合内部运营系统同时使用多端的实际需求。

水平扩展：服务器完全无状态（Access Token 验签不查库），水平扩展无需 Redis 会话共享，仅 `session_version` 查库一次。

应急响应：改密或账号冻结可一次性踢出所有设备，响应有效。

安全纵深：JWT 签名 + session_version + bcrypt + 旋转刷新 + 锁定机制多层防御。

5.2 局限性与缓解措施

局限性	说明	建议缓解措施
无法精准踢出单设备的 Access Token	JWT 无状态，访问令牌在 TTL 内无法主动失效	缩短 Access Token TTL（建议 ≤15 分钟）
无设备管理功能	用户无法查看"当前登录设备列表"，也无法主动踢出指定设备	Phase 2 可补充设备管理 API（依赖 `device_info` 字段）
`refresh_tokens` 表持续增长	每次登录新增一行，依赖定时清理任务	确保定时任务（`deleteExpiredAndRevoked`）正常运行，监控表行数
无并发设备数上限	同一账号可无限台设备登录	内部系统可接受；如有合规要求可加 per-user 行数限制

6. API 端点速览

方法	路径	鉴权	说明
POST	`/api/auth/login`	无	邮箱+密码登录，返回令牌对
POST	`/api/auth/refresh`	无（用 refresh_token）	旋转刷新令牌
POST	`/api/auth/logout`	JWT	撤销当前端 refresh_token
GET	`/api/auth/me`	JWT	获取当前用户信息
POST	`/api/auth/change-password`	JWT	改密并全端下线
POST	`/api/auth/forgot-password`	无	发送 OTP 验证码
POST	`/api/auth/reset-password`	无	OTP 验证码重置密码

7. 文件索引

文件	职责
`backend/lib/core/middleware/auth_middleware.dart`	JWT 验签 + session_version 校验
`backend/lib/core/middleware/rbac_middleware.dart`	角色权限校验
`backend/lib/modules/auth/services/login_service.dart`	登录/刷新/登出/改密核心业务
`backend/lib/modules/auth/repositories/refresh_token_repository.dart`	refresh_tokens 表 CRUD
`backend/lib/modules/auth/controllers/auth_controller.dart`	HTTP 路由处理器
`flutter_app/lib/core/api/api_client.dart`	Flutter 端 Token 注入与自动刷新
`admin/src/api/client.ts`	Admin 端 Token 注入与刷新队列
`app/src/api/client.ts`	uni-app 端 Token 注入与自动刷新